Search in video
0:00
Hi everyone, welcome to observability guru. Today we going to discuss data dog
0:06
bits AI S sur which is one of the hotly released features of data dog. It's a AI
0:13
agent which will allows you to troubleshoot and automatically
0:19
obtain root causes. So we're going to test this feature with uh multiple
0:24
scenarios and as usual I'm going to use uh our application the car rental
0:29
application where uh the functionality is same if it is if you are following uh
0:35
our channel where we have location service which is getting the data from location microser
0:41
detail is coming from the car service and the pricing details are coming again from the car service and we have a
0:48
reservation service as well. So if I go to uh the EC2 instance which
0:55
is which is hosting this I can look at
1:00
the services and you can see uh this is the
1:07
reservation service this is the car service and we have the location service
1:12
uh and everything as well. So uh I already have data dog up and
1:19
running uh for uh this uh set of application. So if I go to APM and if I
1:24
look at services uh you can able to see the services
1:29
start appearing and if I go to logs uh most of the logs are start coming into
1:36
uh the data dog as well. So data dog typically has all the logs metrics and traces. So you can see everything is
1:44
coming. We have couple of monitors by default being created but I created the
1:49
one monitor which will alert us in case uh if there's a increase in uh uh errors
1:57
into location service. So this is what we going to check right so location

2:02
service is getting triggered when we go to this page. So the data will be
2:08
populated from here. So we'll try to simulate a error here.
2:17
Okay, the first error I'm going to simulate is let's do the easiest thing. Let's kill this location service
2:24
so that it will not generate this and let's go to our site. Let me just
2:31
refresh the page and let's start generating some traffic. So other
2:36
services working fine but you will see location service data is not coming.
2:43
Yeah, again I'll start generating couple of traffic until uh we see the alert
2:50
getting fired.
2:56
Well, I created a one alert uh because uh most of these alerts errors now looks
3:03
like been getting captured in API gateway. So we I create a alert in API
3:09
gateway to alert us in case if there if it is encountering errors right. So all
3:15
the location service all those failures will be first captured in here. So we see this alert already been fired. So uh
3:24
let us uh so one way you can start using is now
3:30
the alert is fired. You want to know the root cause. Let's go and ask bits sr to
3:35
investigate. So you can click it here and from here you can trigger an investigation.
3:44
So bits a AI S is an AI agent which is
3:51
been package part of data do. So it has its own way of running. So what it does
3:58
is initially uh it will get the uh the details of the alert. So it will uh
4:04
consume the payload understand what is the uh the all the payroll related details and then it will go through what
4:12
it calls start investigation where it will try to understand uh the highle
4:19

details why why these alerts got triggered. Generally data dog goes through a very
4:24
methodical process where it will first check whether uh there are any run books and it will do a accountwide search and
4:32
it will try to see uh the historical failures as well and based on all of
4:37
this data it will try to understand uh uh the failure scenarios and once the
4:44
failure scenarios are identified it will uh try to validate some of these failure
4:51
scenarios. So the failure scenarios generally it's called as hypothesis. So
4:56
it will try to validate this hypothesis and uh with the existing telemetry data.
5:03
Now here you can see uh it has done the initial investigation and uh what it has
5:10
done uh it uh uh from the in so as I said it will look at
5:17
the runbook details. So at the moment we have not given any much detail in the
5:22
runbook. It was just a blank alert. Nothing in the uh alert body. But this is something which we can improve in
5:28
future. So we can uh provide runbook details and then we can provide uh it it
5:34
will uh look at historical failures. So this is the first time we are using this in this account. There was no failure
5:41
but still it will look at the memory and it will do a general search and with this it came up with initial finding. So
5:48
what it says is backend connection failures cause connection refuse errors on API location in API gateway causing
5:57
500 responses. So it it it is looking at this uh uh failures right. So while it's
6:05
happening let's also generate couple of more traffic. So the failures is still there. So we
6:12
can see uh so it it does it identify the reason right. So uh if you look at that particular alert which we have created
6:19
uh let's go and check uh uh the red alert.
6:25

So the red alert is uh for uh any API gateway fail related failures. We are so
6:32
at the moment it has turned green because there are no failures now. But let's go and uh look at that uh
6:41
alert. So what that alerts uh means uh is if
6:48
there's even a single failure within a 5 minutes per f minute period, right? So
6:54
if there are a single errors within API gateway, this will trigger. So this is just alert and we don't have anything in
7:00
the body. It's just a like a title again repeated in the email body. Not a great
7:07
alert but we are trying to see what bits SR is able to do. So I think it we have
7:13
to be little happy within the first uh uh the uh initial finding itself. It was
7:20
able to isolate it to / API location. Right? So it came up with the initial
7:25
finding. So which is good and then it created a hypothesis. So it's created lot of hypothesis
7:34
failure scenarios. Uh it it went uh with those failure scenarios and start
7:40
validating these scenarios and finally it came up with its uh root cause. So
7:46
what it's saying is backend refuse connection errors. The root cause of the
7:51
API gateway race is that backend service uh so this particular service was not
7:57
accepting connection resulted in connection refuse error. So it's saying it service is not accepting uh the
8:04
request. So the it's API location endpoint based evidence. So it has given the
8:11
evidence saying uh finish and all those things. It has given uh uh the trace uh
8:20
related to these failures as well. We can go and check uh this particular trace.
8:31
So it's able to understand it's related to API location and so uh this is the
8:38
way uh it's having the error connection refused.
8:45
So uh it's able to give that uh backend connection refused. Uh so all the

8:52
hypothesis are related to uh the back end detail. So so it was able to identify that
8:59
right. So let's ask we can even have a conversation with the bit with say we can say uh so what is the exact
9:11
root cause so we can ask uh bitsai uh if you are
9:18
not clear so what is that exact root cause so it's saying in the backend service it was not accepting
9:24
uh and that is the uh the main reset.
9:41
So this I see sometimes this UI issue happen where it's sometimes little difficult to uh go up
9:51
but if you see it says not telemetry available on itself. Okay. So how let's
9:57
ask like how to fix it.
10:08
So it's saying uh okay probably restart this service.
10:16
Uh so it's uh 903 restart the backend service if uh all right.
10:34
Okay. So we this service we know location. So we can ask is location
10:41
service is up and running.
10:46
So that's a question we can ask. So it is coming and say I don't have direct
10:51
access to check uh status of the back end. How I can help you check API
10:58
locations from yeah from investigation API location
11:04
request because it couldn't connect. They strongly suggest that backend service likely location service was down
11:12
or not accepting connection. I cannot confirm its current status because there
11:18
are no data do telemetry from backend host itself. No monitors or
11:24
investigation. Would you like me to query recent trace logs for
11:30
Okay. So what we can check is uh okay it's it's not able to figure it out
11:36

whether uh so we what we can do is we can ask whether can you check location
11:43
service processes up and running.
11:50
So that that's that's one of the key thing. Uh so it says I don't have ability to SSH host or check running
11:57
processes. My capabilities are limited to cing data dock telemetry locks traces
12:02
metric and performing actions. All right. Okay. So so the good news is
12:10
it was able to isolate that this issue is related to API location. So which which is uh from my side I think uh it's
12:17
good uh uh but let's try to see right how how we can improve it. Uh let's say
12:24
can you check what are the processes running
12:29
on this EC2 host.
12:38
Okay. So it's saying it it I can't check the I don't have SH. So I think like
12:43
that's probably one of the the uh capabilities which is currently
12:50
missing because if you see if I go to host infrastructure host and then
12:56
hostess have the ability of looking at all the the processes.
13:03
I go to host and then I should be able to see uh the host detail here.
13:14
I should be able to see the host detail here and uh pretty much I should be able
13:20
to see all the running processes as well. So at the moment uh if I search for location
13:30
it's not running. Let's tell Bits AI can you do that? Let's say this is running
13:36
on this is running on this one.
13:46
Can you check and post
13:54
the processes? I need to
14:01
talk of this post check
14:09

is anything called location service. So we'll try to help
14:15
and see whether it's able to do that. Yeah. So it's uh that's probably a
14:21
limitation but I think we should be happy. uh generally it looks like it has
14:27
worked fine. So if you see uh it look at the uh the error and then it did the
14:34
initial investigation. It was able to understand something wrong with the API location uh even though the alert came
14:41
related to API gateway and it did uh it did hypothesis. It went through some of
14:48
the failure scenarios and then it came up with uh nice root cause. Right? So
14:53
says it's not accepting request. So this is good. Um let us uh look for another
15:02
scenario. Okay. What I did now is I going and
15:08
restart all the services. So API location, API gateway, location service,
15:13
car service, reservation, everything is up and running. So this is good. So this
15:18
will allows me to check our application. Uh yes, everything is working fine. So
15:26
let's let me do one thing, right? Let me probably disable a port or something and
15:31
let's try to uh create some alert. So what I did is I I actually go and
15:38
delete disable the MySQL port. So ideally speaking everything should stop
15:44
working. So I'm not getting any other things.
15:51
So everything is not working.
15:59
Let's try to check whether it this generate alert and
16:06
whether we can use bits s to investigate this as well.
16:14
Yes. So we have couple of alerts already there. We have another API gateway
16:19
error, another location service error. So let's do one thing. Let's uh uh
16:26
investigate both together. Let it run. So uh let me go first to check uh what

16:34
is the error for uh increase in API gateway errors. Let me ask bits s go and
16:41
investigate this. So there's this another error
16:47
which is about increase in location service related uh failures. So I'll
16:53
just go and ask bits s investigate this.
16:58
So this is what we will do. So uh let's monitor one right. Let's start uh uh
17:05
looking at the increasing API gateway request. So as usual it's uh look at the
17:11
uh the alert and it start doing the initial investigation. As I said bits it
17:17
will look at your runbook and run books are detailed usually pulled up. You'll have to embed them part of your uh email
17:26
alert body. So at the moment I have not put anything. So probably that's one reason why some of our root causes are
17:33
not that great. But uh let's see right. Let's see. Let's let's wait and see. And
17:38
then it will look at the memory the past uh recalling past investigation from memory and it will do a general search
17:45
right. So this will get you uh the initial detail. So this is what uh bits
17:51
SR is doing to obtain the initial investigation. So I'll pause the video
17:56
for a few minutes until it comes with the uh the the investigation.
18:05
Great. it start uh summarizing the detail. Now you can see uh let this
18:11
click this view right. So it's look at the runbook, right? Uh so it was able to
18:18
found that uh there's no runbook but it it looks
18:23
pretty decent, right? It was able to say JDBC connection failure in car service. Uh so there were errors API related to
18:32
MySQL exceptions. So it was able to uh get some of this detail.
18:40
So what is the initial finding? So initial finding is car service shows

18:46
JDBC connection failure with uh connection pool timeout errors reservation. So all those JDPC errors
18:53
right $500. And then uh let's go and see it's
18:58
creating a hypothesis. So what are the hypothesis? It's assuming okay is there
19:03
a failure with JDBC connection failures or is there a MySQL connection failure
19:09
or whether there's a JDBC connection failure right so it's then it will start going and validating it right so what
19:16
once the hypothesis or the failure scenarios are created the data dog bits
19:22
s will use the x16 telemetry data it's have access to and go and try to
19:27
validate invalid invalidate or whether data is inconclusive to make a decision.
19:33
So at the moment it's doing going through the validation phase. Let's wait and see.
19:42
It's still going through this hypothesis validation detail. You can see it's
19:48
looking at uh one of the hypothesis as JDBC connection failure. So that was
19:53
kind of like validated. there were
19:59
other hypothesis which is uh kind of like being validated as well. So let's
20:05
see. So this is a very nice view right. So it if you can see it is looking at
20:11
JDPC connection failure and uh it's checking whether MySQL database is uh
20:17
whether it's responding properly whether it's a network latency between uh the application and the database and all
20:25
those details. So this is very good real time it was able to understand some of
20:30
the detail and now it's finished its hypothesis and it came up with final conclusion. So it says car service to
20:38
MySQL network failure cause pole extortion leading API gateway root cause
20:43

of API gateway spike is network connected failure between car service and MySQL database. So it's good. So it
20:50
was pretty much able to understand and navigate that there's a connection
20:56
failure between the application and the database which is good and and then it
21:01
has provided all the uh the errors as well. So you we can quickly go
21:09
to the faces. So it says okay the fine car details uh it's uh having the error.
21:16
So the error is also it's identify those unable to acquire database connections
21:23
and so this is good right like so it was able to identify root causes it isolated
21:29
but let us ask right how to fix this issue.
21:39
So this is the the beauty of uh uh this uh we have that option of uh having a
21:45
conversation Okay. So it's giving me couple of uh
21:53
this is what where I don't really like it where I think it's definitely have a UI issue which uh so let let us do one
22:02
thing right let me just copy this okay I
22:08
finally able to go up so based on here's what's happening and how to fix it root
22:13
cause the API gateway errors are caused by network connected failure between your backend server
22:19
and MySQL database. So all those services failed to acquire JDBC
22:24
connection due to communication link failure. Okay. Check the MySQL database
22:32
availability. Verify MySQL database instance is running and reachable from your application pod host.
22:43
Check the network connectivity from the host.
22:53
restart affected services. So it was able to give some longterm
23:00
solution as well. Uh I start testing right. So it was
23:07

asking us to check whether uh do a test
23:12
and see a tinet. So let let let let us just do and see.
23:20
Okay. So because I know for sure this was not working and trying. So we will
23:26
uh probably probably we'll give this detail
23:33
uh to agent and ask
23:48
Can you help confirm hanging? Yeah.
23:56
Can it confirm hanging required? Next step check if MySQL server is running on the host.
24:07
Access. Let me get this closer to here.
24:14
Yeah. So, MySQL is running. So,
24:20
port is listening.
24:28
So, check security group firewall rule. Verify that my SQL security group allow
24:36
inbound traffic. This is the one I did. So I actually disabled
24:42
this is from traffic group.
24:48
Let me just put everything into this chat.
24:58
So it's interesting. So it's guiding me. So good finding my SQL is running
25:03
listening on the port. So
25:09
okay so it was able to a little bit smartly figure it out. There are some
25:14
issue with the the security group.
25:23
Okay. So let's do one thing right. Let me just go inside. I'll
25:32
I'll I'll check my security group details and share with big AAI.
25:41
Okay, let me share this. This is
25:55
okay. it it was able to identify uh that port 1336 is missing from
26:04

security group and that is the root cause. So it it so idea is that it it
26:10
was uh uh didn't go into the deep and just tell me exactly yeah the port was
26:16
not missing but uh it was able to identify allows me some detail without I
26:24
spending too much time uh allowed me to isolate this issue. So this is good. Uh
26:31
so let's see this is good and let we also there was one alert another alert
26:36
came in which is about spike uh the traffic. So I think it should increase
26:42
in location services. It should have gone to uh something very similar.
26:48
So what was the root database connection failure right? So it was able to isolate
26:54
and then that was a hypothesis and then finally it's gone
27:00
and say root cause of the alert is suddenly spike in database connection failures right so uh I'm not too sure
27:07
right I don't think it's able to have now that agent look at one alert and
27:13
identify the root cause but now whether it's sharing the detail but uh let's ask
27:19
is minus M
27:26
down or what is in excessive?
27:32
Okay, so we asked a pointed question.
27:40
The investigation could not confirm whether my skill was down, right? So, uh that that that's probably doesn't have
27:46
much detail.
27:53
So, uh, that's interesting. So, let's go a little up.
28:05
So, let's go a little up. Based on the investigation finding, here's what we determine. MySQL server down. The
28:12
initial could not confirm whether service actually down or crashed. There are no serverside logs or anything. So
28:19

that's probably something which we should be able to provide later. Network port inaccessibility validated. The
28:26
evidence is strongly point network level connectivity failure. So it was a
28:33
and it says the root cause MySQL port
28:38
unreachable. Okay. So it's again uh close. So
28:46
that's good.
28:51
I added the port now. So ideally I should be able to see things start
28:58
working now. Let's see.
29:05
Yeah, start working fine now. So this alert should be disappeared in short
29:12
time. So uh in nutshell bits AISR is doing a decent job. It was able to
29:18
identify some of these issues
29:23
relatively well and it was able to uh
29:28
provide us uh direction where to investigate. So
29:34
that's the uh the beauty of bits. So uh I can go back again.
29:45
So here uh once you go to B S agent
29:51
you should be I should be able to see now the the test which we have done and
29:56
it will able to show provide me with the report of uh
30:02
the analysis. So the beauty is that a meantime to conclusion those three alerts even though it was not able to go
30:09
deep but it was able to provide me a direction within 3 minutes right within probably 3 and 1/2 minutes which is very
30:16
powerful. So by the time when your on call team members join they not only see the alert but they can now have some
30:23
investigation which will drive them for the root cause.
30:29

So this is really good right. So next what we'll try to do is we'll try to
30:35
create the similar alert uh but let's try to go and improve uh our
30:42
uh the agent our alerts. So we'll provide more context to the agent and
30:47
see whether it's able to uh identify some of these failures.
30:54
Most of the monitors are green now. So let's go and update uh these two
31:03
alerts and when I say update I'm adding little
31:08
more context so that let's see that whether agent is able to uh go little
31:15
deeper now okay so let me go inside this alert and
31:25
within the application uh within the email the alert body I can
31:31
add uh some detail so I'm adding like I'll give more context to the
31:37
application is consist of front end application and backend services so I give the micros service detail and the
31:43
SQL server details as well and this alert also I'll do something
31:52
similar
32:16
All right. Okay. So now that we have uh uh this probably we'll go and we'll do
32:23
the easiest thing. We'll kill one of those uh
32:31
let's kill the location service again 374.
32:39
Okay, location service is no longer there. We
32:45
go to our application. So reservation is coming, pricing
32:53
details are coming but no location service. Let us create some more
32:58
traffic. So hopefully this should uh uh create a alert in our monitor.
33:10
Let's check whether let's wait for some time whether this alert is getting
33:16
generated. So we'll see.

33:26
Okay. I think alert is getting generated already. Not yet. We'll wait.
33:35
Okay. Okay. Okay, so we have our alert now. Let's go to the alert
33:42
and yes, so there's this alert. So let's get bits to investigate. Let's see. It's
33:49
a uh how it's uh performed now. So we try to we recreated a very similar
33:56
alert. We just kill the this micros service and let's see how uh bit AI is
34:06
going to work here. So as usual it's look at the monitor and
34:13
it's trying to do the initial sorry it's trying to do the initial
34:19
uh the analysis. So it's picking the details from the runbook and it's picking the details from memory and it's
34:27
doing a generous search as well. So I'll keep it here uh and and we'll
34:33
see how uh the bits AI is able to pull this data. So while uh it's uh it bits
34:41
AI is doing its work. So one of the thing I want to highlight is if you want to get the best out of bits AI so you
34:47
definitely have to update more detail in the monitor uh providing more context.
34:54
So and and it it not necessarily means that you'll have to put dump everything into the uh the email the alert body you
35:03
can have it in your confluence page and there you can integrate confluence or any other page with data dog. So that
35:09
links you can in integrate uh with the alert. So that way that you can provide more details. So that that's always
35:16
better. The more details you provide and if you see like my monitors are very uh
35:22
basic uh I kind of like not using tagging and other things. So basically I
35:28
probably it's best now you start tagging things properly. Tag it to the proper service and the logs tag it to the
35:34
proper service. So that's way that like it's a you have to even though it's AI

35:40
it doesn't mean that you know it's magically will identify the root cause you will have to still consider it as a
35:46
new team member start training it provide runbook tell it what's your application is doing tell it has a front
35:52
end tell it had backend services tell that backend service are based out of microservices give more detail about the
35:59
microser as well probably give the command how to check whether with the process is up and
36:05
Once you have all those details, it's like a good team member. It will try to do that for you. So now it has done the
36:13
uh the initial investigation. So it's able to identify yes there's a
36:18
connection refused connection failures connection refused errors in API
36:23
gateway. So uh it it was uh it understand like you know why the errors
36:29
are coming from what's the reason for errors. So let's see like whether it was
36:34
it's able to go into detail right. So it's a it's creating the hypothesis. So
36:39
while it's doing that let us create some traffic as well. This is always good. We
36:44
create traffic. We uh keep our bits s guessing as well. Yes.
36:54
So let's go and check. Yes. So let's uh let's give some time
37:01
for bits a s sur agent to go through this. So there are a lot of agents in
37:07
the market. So one thing I like about bits AI SR is
37:13
not only it's just provide you the root cause but it will do that with a certain confident level and that confident comes
37:19
with it will go and try to validate this hypo hypothesis the h the failure
37:25
scenarios with the existing telemetry data and then it will provide you its
37:31
way. So as you can see it came up with the investigation conclusion. So it says
37:37

API errors caused by connection refuse right API connection refuse. So it was
37:43
able to say this is related to get API location endpoint and then it has given
37:50
probably if I go to the traces it's able to say uh
37:57
okay what was this issue? So there's this connection failure errors.
38:06
So there are other options also it's giving. So it says what services what services running on and are there
38:13
passing. So the investigation wrap up what part of would you like to report further? So I can get it to create an
38:21
incident and say create an incident for API gateway connection reviews. I'll
38:26
help you create an incident for this issue. Let me first discover the available actions. Get the details for
38:32
creating the data of the incident. So that that's one one thing like you can we can use it to create incident as
38:41
well
38:51
and say yes.
39:01
So let's see uh agent is able to create the incident. It says bad request.
39:09
Let me retry with yeah it's like a team member right? It might make mistake will
39:14
identify the mistake and then it will just try to uh fix it by itself right
39:21
400 back request.
39:29
Okay. uh create an incident title and you all
39:38
can
39:45
so let's let's try to see whether agent can do that for us.
39:54
Unfortunately, the incident creation action keep failing because your data dog organization required description
39:59

field that isn't supported as a configuration parameter in the available action. This is a limitation of current
40:05
action information. So, yeah, that's fine, right? Like uh okay, so that's
40:12
probably a limitation. So, let's ask uh how to fix
40:18
issue. Are you able to see location? So is my
40:26
process process running on
40:31
post processes.
40:48
So it says the service is either down or crashed
40:53
and it's uh it's doing what it was doing previously. It was giving some command
40:58
to see whether up and running. So let's do that.
41:15
One second. Let's check with what it's saying.
41:22
Okay, let's just share this with our agent and see what
41:28
agent is saying.
41:35
This empty output
41:43
nothing is so immediate check.
42:00
It says uh look at that location service name.
42:15
We'll try to get that location service name from here to it.
42:25
So this is the job.
42:39
to give this output to our
42:52
it simply says yeah probably know just go and start it so it it was doing a
42:59
decent job I wouldn't say like you know it's perfect but I think it's in the right direction
43:06
so that's it that's what we want to go through Thus those are a couple of demos we I want to focus and
43:16

let let us check one or two examples more.
43:24
Okay. So I restarted all my uh services again. So everything is
43:31
good. Let us do a quick check.
43:36
It's loading. Everything is loading. So this is good. So this allows us to uh
43:42
check whether this uh we'll do another one final scenario and see how it is
43:47
going. Okay. So we'll check for one other last
43:53
scenario. So we I have created one alert to look at the API response time uh API
44:00
gateway response time. So it is breaching uh so uh because uh I probably
44:06
ask to get myself alert if it take more than 2 seconds. So let's use bits
44:13
to investigate this. So while we do that let me also create a couple of traffic.
44:28
Uh okay. So it's kicking off uh bits AI SR
44:33
investigation.
44:41
So it's look at the API gateway response uh and then it's looking at the runbook
44:46
memory and general search uh to support u uh to obtain uh initial analysis.
45:01
Okay, it came up with the initial findings.
45:07
So let's just go there and check. So it says high latency API gateway
45:14
location service. It identify the latencies coming from location service
45:20
and also there's something for car service as well and it went through the
45:26
hypothesis and finally it came up with the detail the spiking traffic primary
45:32
to high latent downstream car service tra that get cast request almost 5
45:38
second during wait time on downstream uh while underline my school so no
45:45
errors indicating the bottleneck in the response of the layer of car sides.
45:53

Okay. The latence is mainly caused by drowning car span which particles
46:12
like uh uh
46:21
so we can ask right what is the root cause here.
46:32
So we'll ask that.
46:38
Okay.
47:02
Okay. The API get a response time spike was caused by high latency in car service. So it's looking at 5 seconds
47:09
the bottleneck uh fn find or
47:14
this gap. Okay. So it I think it is it's coming
47:20
closer. uh even it has shared uh what do you call this particular uh it shared
47:27
that particular uh the the tra span as well so it's again
47:35
little decent uh probably we can live with it so uh thanks everyone thanks for
47:40
taking time so today we went through uh live demos of bits sr uh so in order to
47:48
get this the best out of it you'll have to add more uh runbook knowledge to the agent. Make sure you use tagging and
47:54
other things. So treat it as a your team member so you can get the best out of
48:00
it. Thanks everyone. Thank you. Bye.